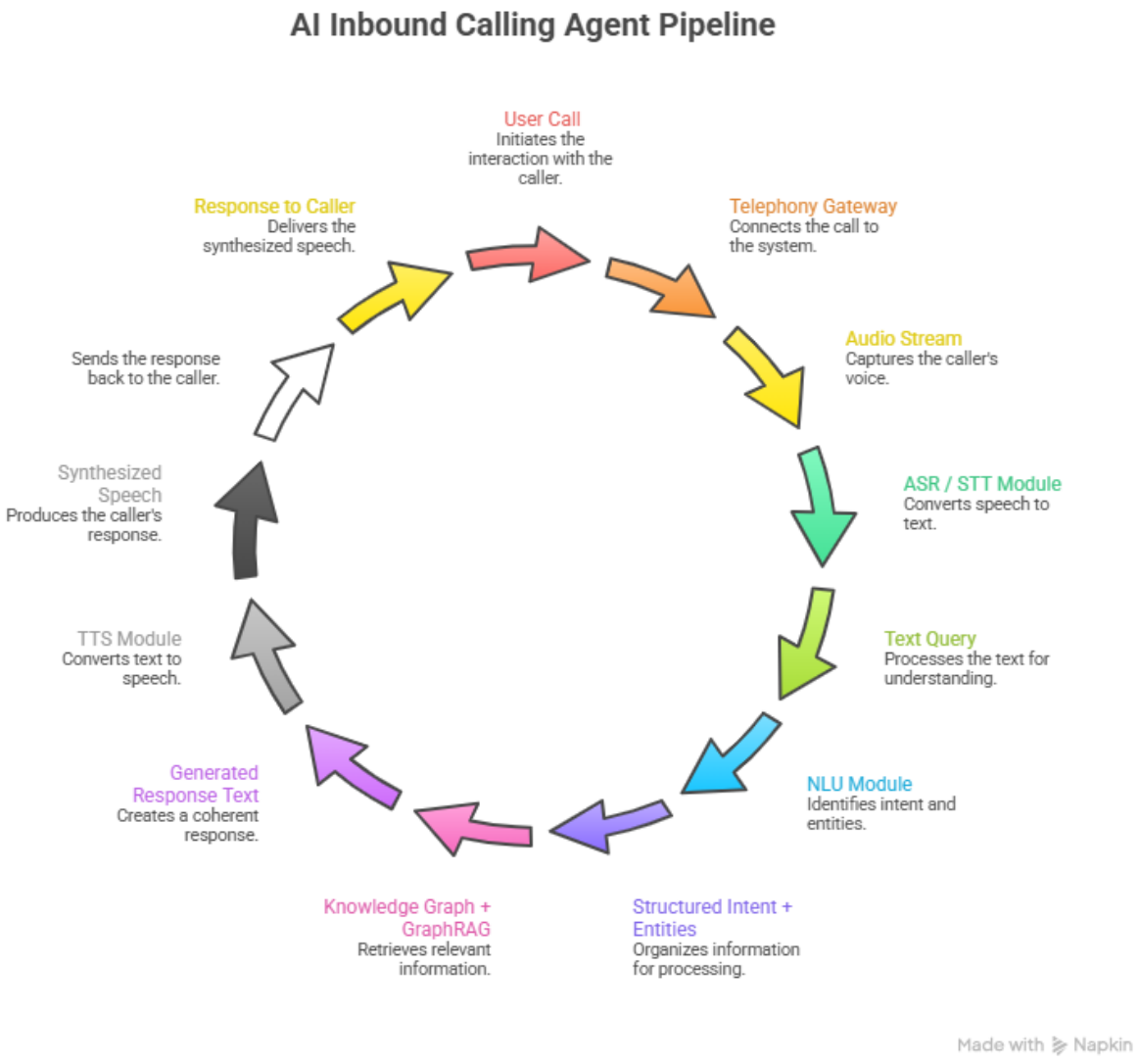
Overall System Pipeline - AI INBOUND CALLING AGENT



1. Module 1: ASR / STT (Automatic Speech Recognition)

Purpose

The **ASR/STT module converts the caller's spoken query into machine-readable text**.

This module handles **speech recognition and transcription** of the incoming audio stream.

Input

The ASR module receives:

Audio Stream

Captured from the telephony gateway.

Typical formats:

- *WAV*
- *PCM*
- *FLAC*
- *OPUS*

Example input:

Audio: "Mujhe BS Computer Science ki eligibility bata dein."

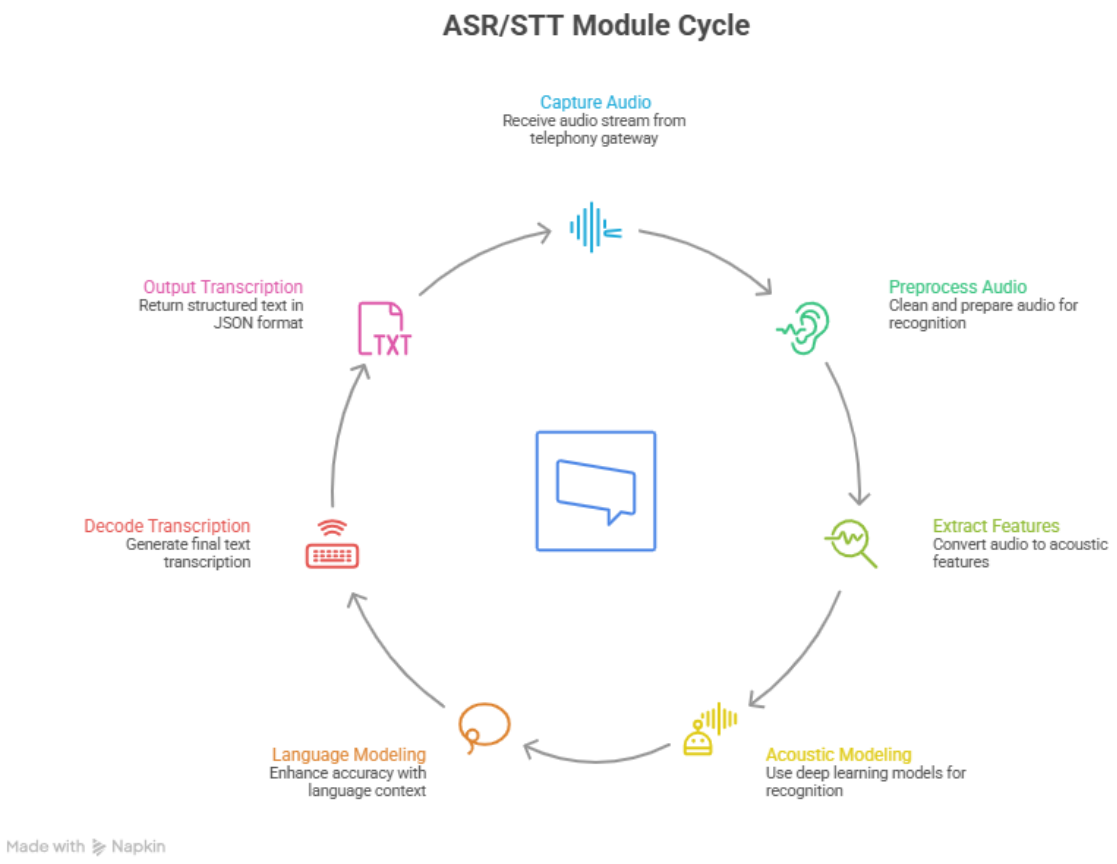
The audio may be streamed in **small chunks for real-time processing**.

Internal Processing

ASR engines perform the following steps:

1. **Audio preprocessing**
    - Noise reduction
    - Voice activity detection
    - Silence trimming
2. **Feature extraction**
    - Convert speech signal to acoustic features
    - MFCCs or spectrogram features
3. **Acoustic modeling**
    - Deep learning models (CTC, RNN-T, Transformer)

4. **Language modeling**
    - Improves transcription accuracy
5. **Decoding**
    - Generates final transcription text



Output

The ASR module returns the **transcribed text of the user query**.

Recommended format: **JSON**

Reason:

- Structured
- Easy for NLU to parse

- Extensible (confidence score, timestamps)

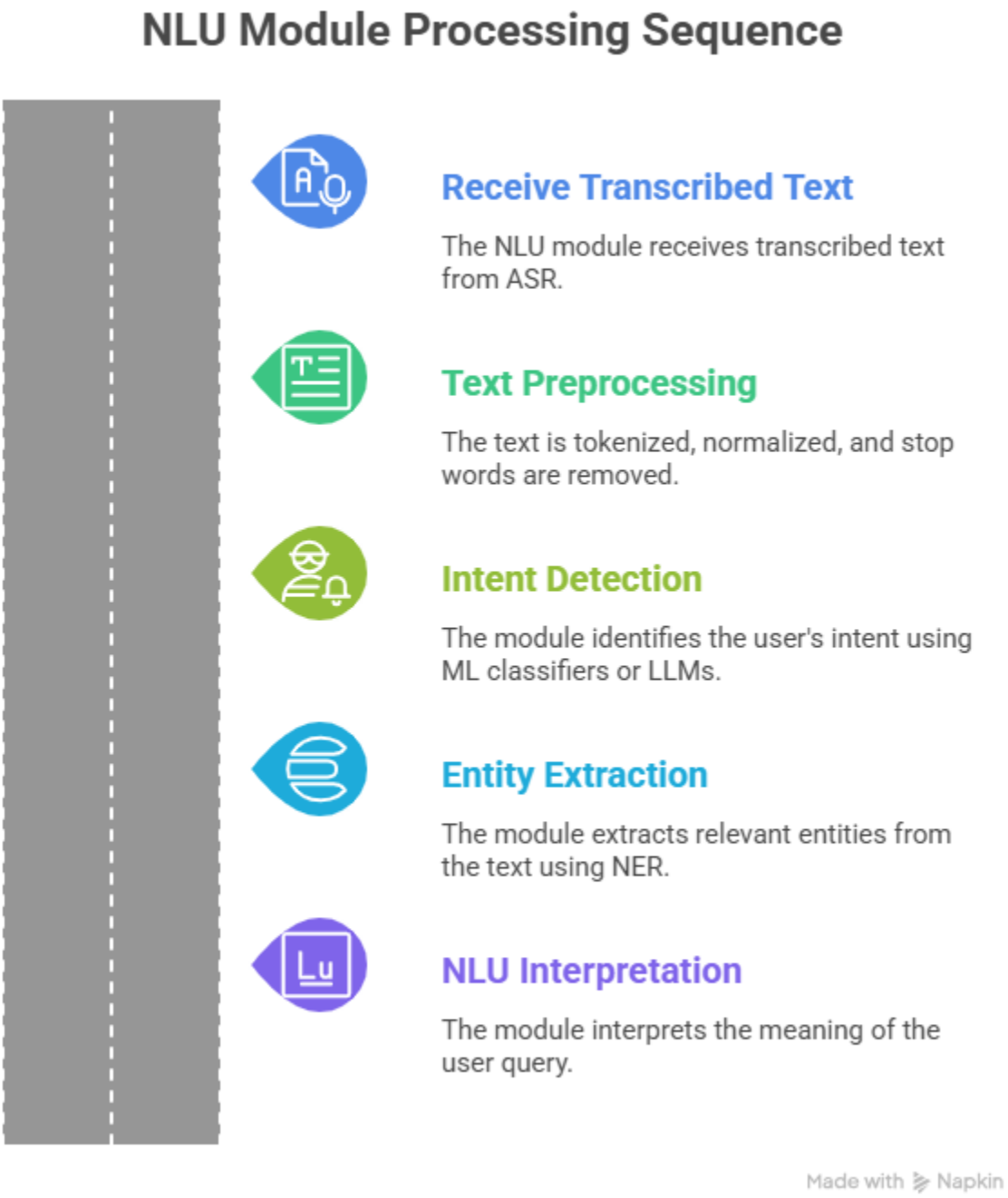
Example output:


```
{
  "transcription": "BS Computer Science ki eligibility kya hai",
  "confidence": 0.93,
  "language": "ur",
  "timestamp": "2026-03-06T10:20:00"
}
```


Where Output is Used

The **NLU module consumes this transcription** as input to understand the user's intent.

2. Module 2: NLU (Natural Language Understanding)



Purpose

The NLU module interprets the **meaning of the user query**.

It identifies:

- **Intent** (what the user wants)
- **Entities** (important keywords)

Example:

User query:

"BS Computer Science ki eligibility kya hai?"

NLU interpretation:

Intent → ask_program_eligibility
Entity → program = BS Computer Science

Input

The NLU module receives **transcribed text** from ASR.

Example:

```
{
  "transcription": "BS Computer Science ki eligibility kya hai"
}
```

Internal Processing

The NLU system performs:

1. Text Preprocessing

- Tokenization
- Normalization
- Stop-word removal (optional)

2. Intent Detection

Using:

- ML classifiers
- Transformer models
- Prompt-based LLM classification

Example intents:

ask_program_duration
ask_program_eligibility
ask_program_courses
ask_course_prerequisite
ask_semester_courses

3. Entity Extraction

Named entity recognition identifies relevant entities.

Example entities:

ProgramName
CourseCode
SemesterNumber
DepartmentName

Output

The NLU module produces **structured semantic data**.

Recommended format: **JSON**

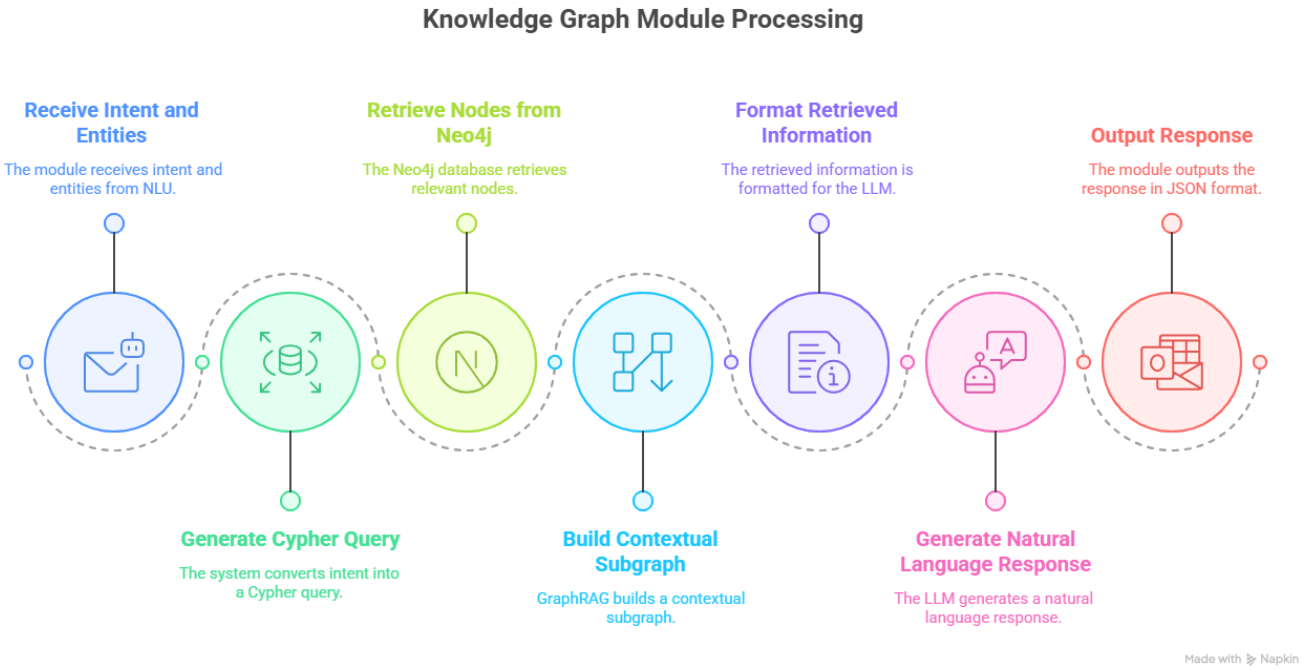
Example output:

```
{
  "intent": "ask_program_eligibility",
  "entities": {
    "program": "BS Computer Science"
  },
  "confidence": 0.92
}
```

Where Output is Used

The **Knowledge Graph + GraphRAG module consumes this structured query** to retrieve relevant knowledge.

3. Module 3: Knowledge Graph + GraphRAG



Purpose

This module retrieves **factual information from the knowledge graph** and generates a contextual response.

It combines:

1. **Knowledge Graph querying (Neo4j)**
2. **GraphRAG reasoning**
3. **LLM-based response generation**

Input

The KG module receives **intent + entities from NLU**.

Example input:

```
{
  "intent": "ask_program_eligibility",
  "entities": {
```

```
    "program": "BS Computer Science"
  }
}
```

Internal Processing

The KG module performs three main steps.

Step 1: Query Generation

The system converts the intent into a **Cypher query**.

Example:

```
MATCH (p:Program {name:"BS Computer Science"})
RETURN p.eligibility
```

Step 2: Graph Retrieval

The Neo4j database retrieves relevant nodes.

Example result:

```
Program: BS Computer Science
Eligibility: FSC Pre-Engineering or ICS
Duration: 4 years
```

Step 3: GraphRAG Reasoning

GraphRAG builds a **contextual subgraph**.

Example:

```
Program
  ↓
Department
  ↓
Courses
  ↓
Prerequisites
```

The retrieved information is formatted and passed to the LLM for natural response generation.

Output

The KG module produces a **natural language response**.

Recommended format: **JSON**

Example:

```
{
  "response_text": "The eligibility for BS Computer Science is FSC Pre-Engineering or ICS with required merit criteria.",
  "source": "KnowledgeGraph"
}
```

Where Output is Used

This response text is sent to the **TTS module** for speech synthesis.

4. Module 4: TTS (Text-to-Speech)

Purpose

The TTS module converts the **system-generated text response into speech** so the caller can hear it.

Input

The TTS module receives **response text**.

Example:

```
{
  "response_text": "The eligibility for BS Computer Science is FSC Pre-Engineering or ICS."
}
```

Internal Processing

The TTS system performs:

1. Text Processing

- Normalization
- Pronunciation modeling
- Prosody generation

2. Speech Synthesis

Deep learning models generate audio:

Examples:

- Tacotron
- FastSpeech
- VITS

3. Audio Post-processing

- Voice modulation
- Speech pacing

- Natural pauses

Output:

The TTS module produces **audio speech output**.

Recommended format:

WAV
PCM

Example output:

response_audio.wav

Where Output is Used

The audio is returned to the **telephony gateway**, which plays the response to the caller.

Recommended Data Format Between Modules

Module	Input Format	Output Format	Reason
ASR/STT	Audio (WAV/PCM)	JSON	Structured transcription
NLU	JSON	JSON	Intent + entity structure
KG + GraphRAG	JSON	JSON	Response generation
TTS	JSON	Audio (WAV)	Speech playback

Final Structured Pipeline with Formats

User Speech

↓

Audio Stream (WAV)

↓

ASR/STT

↓

JSON (transcription)

↓

NLU

↓

JSON (intent + entities)

↓

Knowledge Graph + GraphRAG

↓

JSON (response text)

↓

TTS

↓

Audio (WAV)

↓

Caller hears response